UNIVERSITAT JAUME I

Máster en Sistemas inteligentes — Trabajo Fin de Máster

Development and Implementation of a Person-Following System on a Mobile Robot — Part 2 Extension

Memoria del Trabajo Fin de Máster

Autor: Juan Muñoz Rovira

Tutor: Enric Cervera

Curso 2025–2026

Nota sobre el estado de este documento

Esta memoria tiene siete capítulos y se ha escrito a partir de lo que hay en el repositorio del proyecto y de los datos de las sesiones de laboratorio. La última sesión, el 23 de septiembre de 2026, validó el gesto “casa” (§7.8). El Capítulo 7 recoge las pruebas hasta la fecha de cierre, y lo que queda fuera del alcance del TFM aparece como trabajo futuro (§7.10).

Índice

Nota sobre el estado de este documento 1
Índice 2
Capítulo 1 — Introducción y planteamiento del problema 4
1.1 Contexto y motivación 4
1.2 Objetivos del TFM 4
1.3 Alcance y limitaciones 5
1.4 Planificación 5
1.5 Estructura de la memoria 5
Capítulo 2 — Arquitectura del sistema 7
2.1 Visión general 7
2.2 Diagrama de nodos y topics 7
2.3 Descripción de cada nodo 7
2.4 Gestión del apagado coordinado 9
2.5 Árbol de transformaciones (TF) 9
2.6 Parámetros de configuración 10
2.7 Dependencias del paquete ROS 2 10
2.8 Arquitectura de Nav2 (Fase 3) 11
Capítulo 3 — Uso de herramientas de inteligencia artificial 12
3.1 Declaración de uso 12
3.2 Herramientas utilizadas 12
3.3 Reflexión sobre el uso ético de la inteligencia artificial 12
3.4 Estimación del impacto 12
Capítulo 4 — Diario de desarrollo 14
Capítulo 5 — Estado del arte 14
5.1 Introducción y alcance del capítulo 14
5.2 Sistemas de seguimiento de personas con robots móviles 14
5.3 Detección de personas mediante LiDAR 2D (detección de piernas) 15
5.4 Estimación de pose humana por visión 15
5.5 Fusión sensorial LiDAR-cámara para detección robusta 15
5.6 Interacción humano-robot basada en gestos 16
5.7 Seguimiento de estado: filtros bayesianos 16
5.8 Evitación de obstáculos y control de movimiento 16
5.9 Cartografía y navegación autónoma (SLAM y Nav2) 16
5.10 Síntesis: posicionamiento de este TFM respecto al estado del arte 17
Referencias 17
Capítulo 6 — Implementación 19

6.1 Introducción y alcance 19
6.2 Estructura del paquete y gestión de configuración 19
6.3 detection_node — clustering, características de pierna y filtrado temporal 19
6.4 visual_detection_node — detección dual y reconocimiento de gestos 20
6.5 tracking_node — filtro de Kalman, control PD y evasión de obstáculos 20
6.6 control_node — máquina de estados finitos y teleoperación 21
6.7 collision_handling_node 22
6.8 user_interface_node 22
6.9 slam_node (experimental, desactivado por defecto) 22
6.10 Despliegue: compilación y sincronización con el robot 22
6.11 Código no registrado en el paquete 22
Capítulo 7 — Resultados y evaluación 23
7.1 Metodología de validación 23
7.2 Configuración experimental 23
7.3 Resultado 1 — Fusión sensorial sin movimiento (25 de junio de 2026) 23
7.4 Resultado 2 — Progresión de correcciones de continuidad en movimiento (8 de julio de 2026) 24
7.5 Resultado 3 — Repeticiones por escenario con el sistema estabilizado (22 de julio de 2026) 26
7.6 Resultado 4 — Escenario de obstáculo: de los primeros contactos a la evasión sin contacto (22–23 de julio de 2026) 27
7.7 Resultado 5 — Nav2: localización y navegación autónoma (27 de julio de 2026) 27
7.8 Resultado 6 — Gesto “casa”: integración de seguimiento y navegación (23 de septiembre de 2026) 28
7.9 Limitaciones de los resultados actuales 28
7.10 Trabajo futuro 29

Capítulo 1 — Introducción y planteamiento del problema

1.1 Contexto y motivación

En el laboratorio de la UJI ya había un TurtleBot 2 que seguía a una persona con el LiDAR, pero no sabía dónde estaba ni cómo volver a un sitio.

Este TFM amplía un trabajo anterior en el que se desarrolló un sistema de seguimiento de personas sobre un TurtleBot 2 (base Kobuki), capaz de detectar y seguir a una persona mediante un LiDAR RPLIDAR A2M8 y clustering (DBSCAN) sobre el patrón de piernas.

El sistema original tenía varias limitaciones, y de ahí sale esta extensión:

- **Sin cartografía: sin mapa del entorno, lo que impedía cualquier navegación planificada.**
- **Sin localización global:** ante la pérdida de la persona, el robot no podía orientarse.
- **Interacción limitada: sin mecanismo claro de inicio y parada del seguimiento.**
- **Sin integración de cámara: la detección visual no formaba parte del bucle de control.**

1.2 Objetivos del TFM

Objetivo general

Extender el sistema de seguimiento de personas hasta convertirlo en una plataforma de asistencia móvil capaz de navegación autónoma, comprensión del entorno y comunicación con el usuario en interiores.

Objetivos específicos

1. **Módulo de interacción gestual. Empezar y parar el seguimiento con gestos, usando MediaPipe Pose (mano derecha para iniciar, izquierda para detener). Tras sustituir la cámara (Logitech C270 → SPCA2650) y ajustar el umbral de visibilidad, ambos gestos se dispararon de forma fiable en pruebas reales. Objetivo completado.**
2. **Cartografía mediante SLAM. Integración de SLAM Toolbox con el LiDAR para construir mapas del laboratorio en tiempo real, reutilizados en localización y navegación. Objetivo completado.**
3. **Navegación autónoma con Nav2. Alcance: AMCL sobre mapa guardado y navegación a un punto objetivo. Una vez resuelto el problema de latencia DDS (bastaba con usar comandos más largos), la navegación alcanzó seis de siete objetivos, incluyendo dos trayectos de ~8 m y la evasión de un obstáculo no mapeado. Objetivo completado.**
4. **Fusión sensorial LiDAR-cámara. La cámara aporta el rumbo cuando el LiDAR no identifica un par de piernas. La validación estática dio un 100 % de detección, pero en movimiento aparecieron saltos y saturación angular, que se resolvieron con un filtro de continuidad, una puerta de Mahalanobis y un límite de velocidad angular, además de corregir un signo invertido en el control PD y en el sector de evasión. Objetivo completado.**
5. **Guiado de usuarios. Que el robot acompañe a la persona hasta destinos predefinidos del edificio. Primer paso validado en el robot real el 23 de septiembre de 2026: con el gesto "casa" el seguimiento y la navegación funcionan juntos en un mismo sistema, con 9 de 11**

vueltas a casa completadas (82 %) y 0,27 m de error medio de llegada (§7.8). El guiado a varios destinos queda como trabajo futuro (§7.10).

6. **Objetivo exploratorio.** Probar códigos QR para que el robot reconozca las habitaciones. Queda pendiente al cierre de este capítulo.

1.3 Alcance y limitaciones

El desarrollo se realiza sobre el hardware disponible en el laboratorio de robótica de la Universitat Jaume I:

- **Robot:** TurtleBot 2 sobre base Kobuki.
- **Unidad de cómputo embarcada:** Intel NUC con Ubuntu 24.04 LTS y ROS 2 Jazzy.
- **Sensores:** LiDAR 2D RPLIDAR A2M8 (alcance de 12 m, 10 Hz) y cámara RGB (Logitech C270 hasta la sustitución por una SPCA2650, motivada por problemas de encuadre del gesto). También se contempla una cámara RGB-D Orbbec Astra, cuya integración queda pendiente.
- **Entorno de prueba:** instalaciones interiores de la Universitat Jaume I.

Se asumen las siguientes limitaciones:

- Navegación restringida a dos dimensiones (planta única).
- Detección de personas limitada al plano horizontal, a la altura de las piernas, por ser el plano de barrido del LiDAR.
- Comunicación entre el PC de control y el robot a través de la red WiFi del laboratorio.

1.4 Planificación

El desarrollo se organizó en cinco fases, según la disponibilidad del laboratorio (sesiones acotadas en junio-julio, sin acceso en agosto). La Tabla 1.1 resume el estado de cada fase.

Tabla 1.1
Fases del proyecto

Fase	Descripción	Periodo	Estado
1	Definición, revisión del sistema previo y redacción inicial	Mayo 2026	Completada
2	Módulo de interacción gestual, integración de SLAM y fusión sensorial	Junio 2026	Completada
3	Navegación autónoma completa con Nav2	Julio 2026	Completada
4	Validación experimental en el entorno de la UJI	Junio–julio 2026	Completada
5	Cierre, memoria final y defensa	Agosto–septiembre 2026	En curso

Las tareas de validación no completadas dentro del calendario disponible se documentan como trabajo futuro en el Capítulo 7.

1.5 Estructura de la memoria

El resto de la memoria se organiza de la siguiente manera:

- **Capítulo 2 — Arquitectura del sistema: nodos, topics y flujo de datos.**
- **Capítulo 3 — Herramientas de IA: declaración de su uso como apoyo al desarrollo.**
- **Capítulo 4 — Diario de desarrollo: registro cronológico del progreso técnico.**
- **Capítulo 5 — Estado del arte: revisión bibliográfica conectada con las decisiones de diseño.**
- **Capítulo 6 — Implementación: algoritmos, fórmulas de control y estructura del paquete.**
- **Capítulo 7 — Resultados y evaluación: análisis de los datos experimentales de la validación.**

Capítulo 2 — Arquitectura del sistema

2.1 Visión general

El sistema se organiza como nodos ROS 2 independientes, comunicados solo por el bus de mensajes (DDS). Así se pueden activar o desactivar nodos, cambiar un módulo (por ejemplo el detector de personas) sin reescribir el resto, y lanzar solo una parte del sistema mientras se desarrolla.

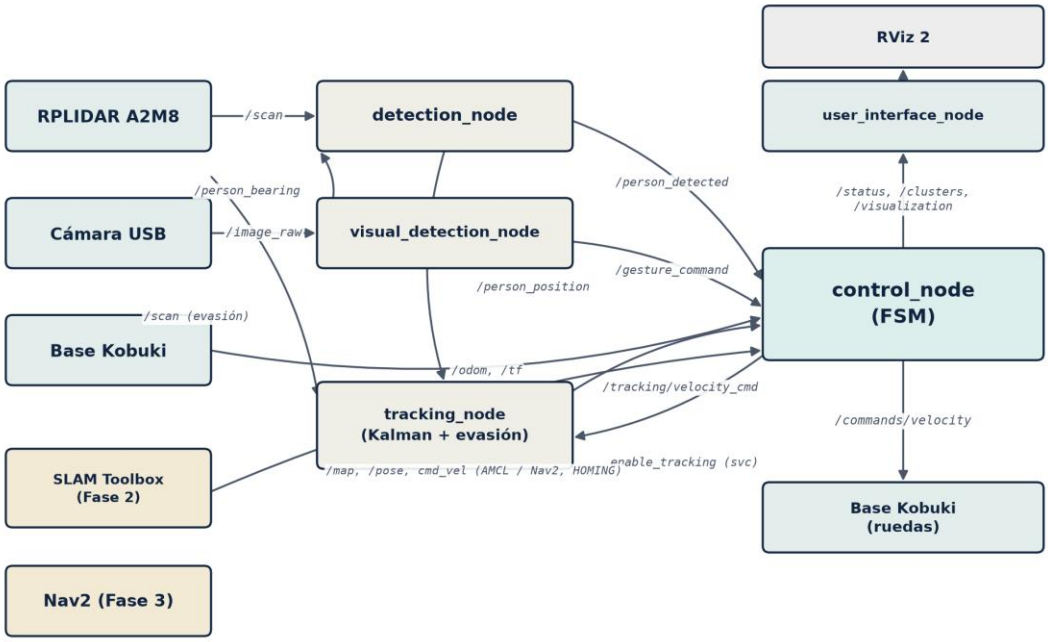
Todo el sistema se empaqueta bajo el paquete ROS 2 person_follower, de tipo ament_python.

2.2 Diagrama de nodos y topics

La Figura 2.1 resume el flujo de datos entre los nodos principales. Desde la Fase 2, visual_detection_node también publica /person_bearing hacia detection_node: el respaldo de fusión sensorial del apartado 2.3.1, activo solo cuando el LiDAR no encuentra un par de piernas válido.

Figura 2.1

Flujo de nodos y topics del sistema person_follower.



Nota. Elaboración propia a partir del código fuente del proyecto.

2.3 Descripción de cada nodo

2.3.1 detection_node

Función: detección de personas mediante el LiDAR.

El algoritmo filtra el ruido del LiDAR por mediana, interpola para aumentar la resolución angular y agrupa los puntos con DBSCAN (eps = 0,12 m, min_samples = 4). Se quedan los clústeres que pueden ser una pierna (por tamaño, radio y forma) y se buscan pares a una distancia razonable.

Como respaldo (Fase 2), sin un par de piernas válido el nodo usa el rumbo de visual_detection_node (MediaPipe) para elegir el clúster LiDAR mejor alineado y publica su centroide. Sin él, el seguimiento se detenía tras 30 s sin piernas aunque la cámara sí viera a la persona. Validado sin movimiento: 100 % de detección, desviación de rumbo ≈6°.

El DBSCAN está reimplementado sobre scipy.spatial.cKDTree, y no sobre scikit-learn, porque la versión de Python del NUC no era compatible con la extensión compilada de esa biblioteca.

Parámetros configurables: distancia máxima de detección (2,5 m, reducida desde 6,0 m), radio de vecindad DBSCAN (0,12 m), tamaño de clúster de pierna (4-200 puntos) y tiempo de espera de cámara (1,0 s). Se suscribe a /scan, /person_detected_visual y /person_bearing; publica /person_detected, /person_position, /clusters/general y /clusters/legs.

2.3.2 visual_detection_node

Función: detección de personas y reconocimiento de gestos mediante cámara.

El nodo procesa cada fotograma con MediaPipe Pose para detectar el esqueleto humano, en lugar del detector HOG original, que necesitaba el cuerpo entero en el encuadre, algo imposible a la distancia real de seguimiento (~1 m).

Los gestos se reconocen por landmarks corporales de MediaPipe Pose (no Hands): mano derecha levantada por encima del hombro inicia el seguimiento, izquierda lo detiene. Cada gesto exige varios fotogramas de confirmación consecutivos y un tiempo de espera entre comandos, para reducir falsos positivos.

El nodo calcula además el rumbo horizontal de la persona desde el punto medio de los hombros, según el campo de visión de la cámara, y lo publica para la fusión de detection_node. Se suscribe a /image_raw y publica /person_detected_visual, /gesture_command, /person_bearing, /pose/keypoints y /camera/image_processed.

2.3.3 tracking_node

Función: estimación de la posición de la persona y cálculo de los comandos de velocidad.

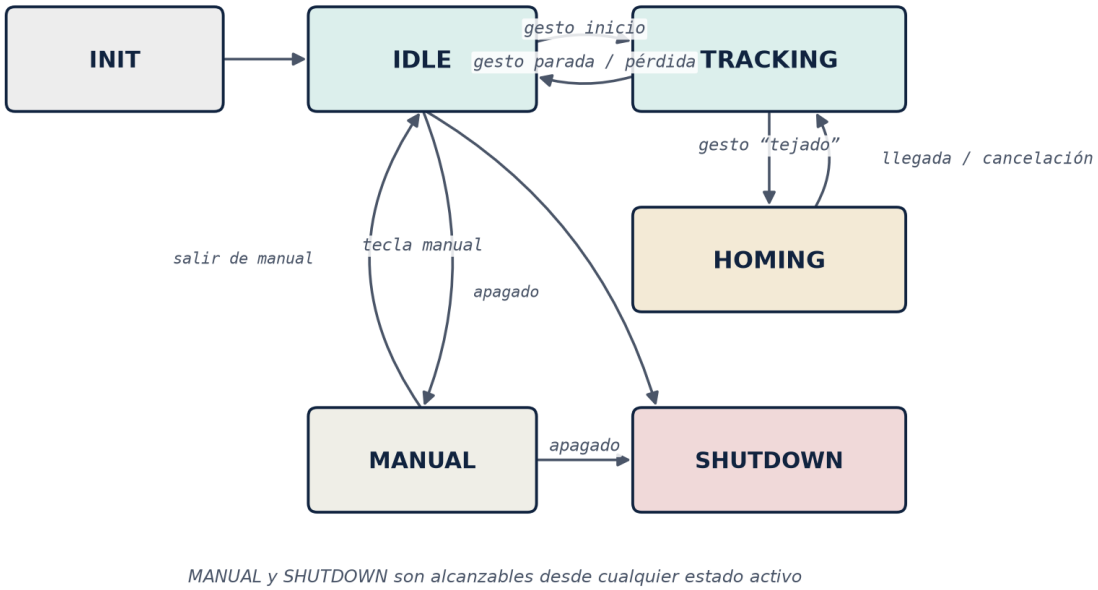
Aplica un filtro de Kalman ([x, y, vx, vy, ax, ay], aceleración constante, §6.5) sobre la posición de detection_node, calculando distancia y ángulo hacia la persona. Implementa evasión de obstáculos sobre el sector frontal ±50° del LiDAR: ante obstáculos a menos de 0,40 m genera repulsión angular y reduce la velocidad lineal, y detiene el robot si la persona desaparece más de dos segundos. Activado por el servicio enable_tracking, invocado por control_node.

2.3.4 control_node

Función: máquina de estados finitos central del sistema.

Implementa la máquina de estados de la Figura 2.2: IDLE (espera de persona autorizada), TRACKING (retransmite las velocidades de tracking_node), HOMING (retransmite la velocidad de Nav2 hacia la pose “casa” tras el gesto “tejado”, §7.8), MANUAL (teleoperación por teclado) y SHUTDOWN (apagado coordinado). Con cámara habilitada, iniciar el seguimiento requiere el gesto; si no, se inicia automáticamente.

Figura 2.2
Máquina de estados finitos de control_node.



Nota. Elaboración propia a partir del código fuente del proyecto.

2.3.5 collision_handling_node

Monitoriza la distancia mínima al obstáculo más cercano y publica una señal de colisión cuando algún punto del barrido del LiDAR está a menos de 0,4 m. control_node no reacciona aún a esta señal; su integración está prevista junto con Nav2.

2.3.6 user_interface_node

Se encarga de la visualización en tiempo real: lanza RViz 2 con la configuración del paquete, publica marcadores (posición estimada, clústeres del LiDAR, marcador del robot) y un estado de diagnóstico periódico.

2.3.7 slam_node (experimental)

SLAM 2D básico, hecho por nosotros en la Fase 1. En la Fase 2 lo sustituyó SLAM Toolbox, que es más robusto, cierra bucles y permite guardar los mapas; el nodo sigue desactivado por defecto.

2.4 Gestión del apagado coordinado

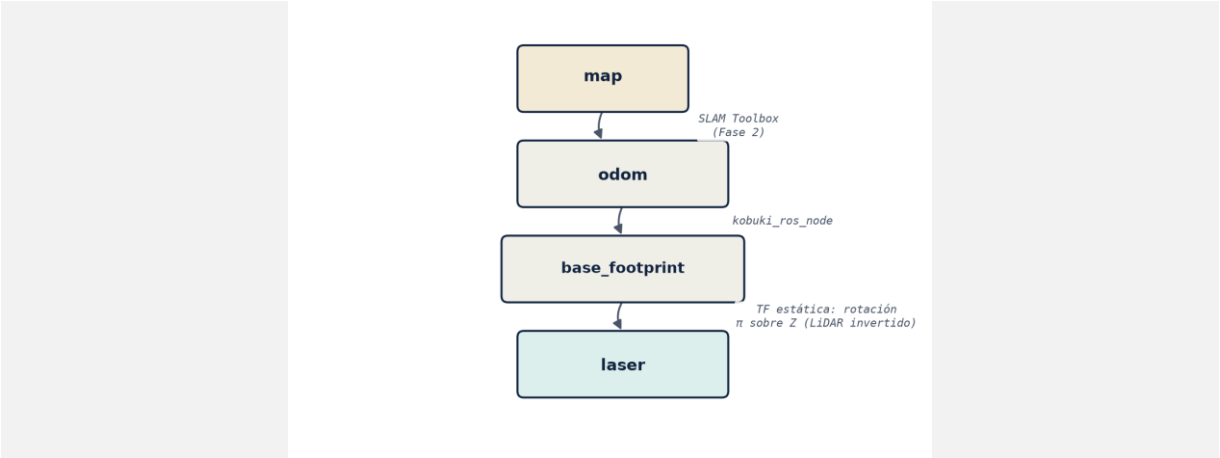
El sistema implementa un apagado coordinado para evitar que el robot quede con velocidades activas: control_node publica una señal de apagado, los nodos suscritos detienen su actividad y confirman la parada, y control_node espera todas las confirmaciones antes de finalizar.

2.5 Árbol de transformaciones (TF)

La Figura 2.3 muestra el árbol de transformaciones. La transformación base_footprint→laser se publica de forma estática, con una rotación de 180° sobre Z, porque el LiDAR está montado invertido.

Figura 2.3

Árbol de transformaciones (TF) del sistema.



Nota. Elaboración propia a partir del código fuente del proyecto.

2.6 Parámetros de configuración

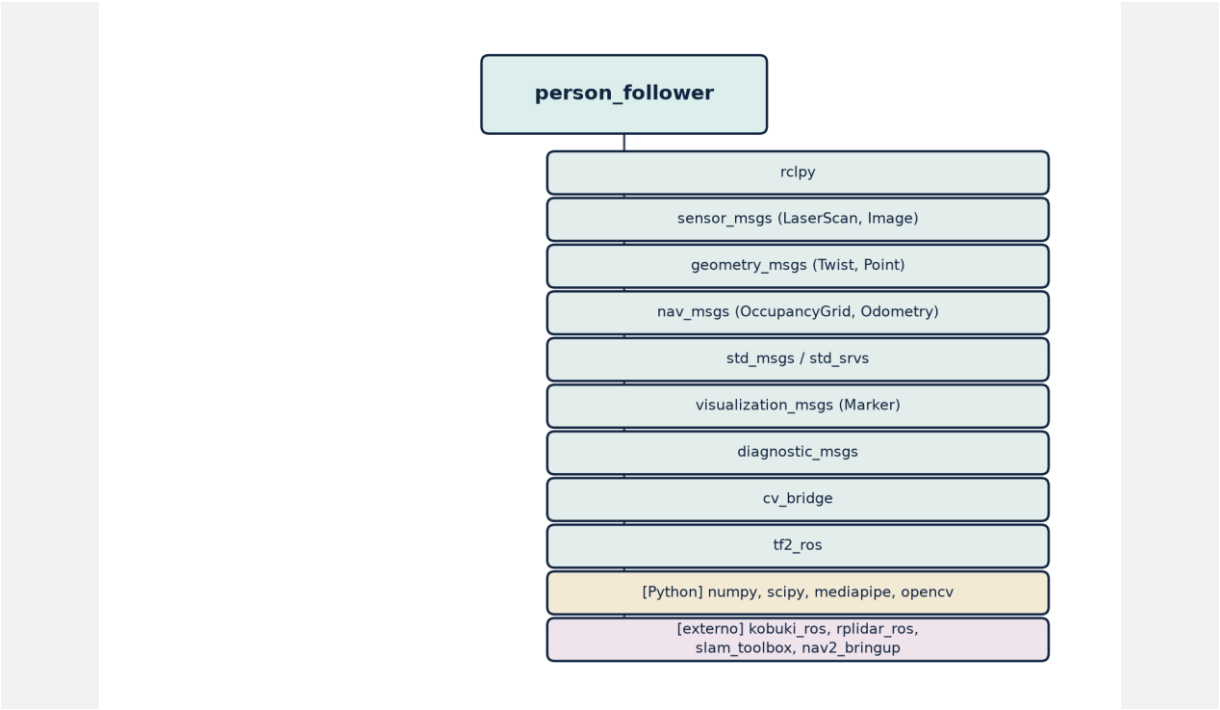
Todos los parámetros ajustables están en un archivo de configuración común, así que se pueden cambiar umbrales, distancias de seguridad y comportamientos sin recompilar. SLAM Toolbox usa un archivo de configuración independiente.

2.7 Dependencias del paquete ROS 2

La Figura 2.4 resume las dependencias del paquete person_follower, tanto internas al ecosistema ROS 2 como externas.

Figura 2.4

Dependencias del paquete person_follower.



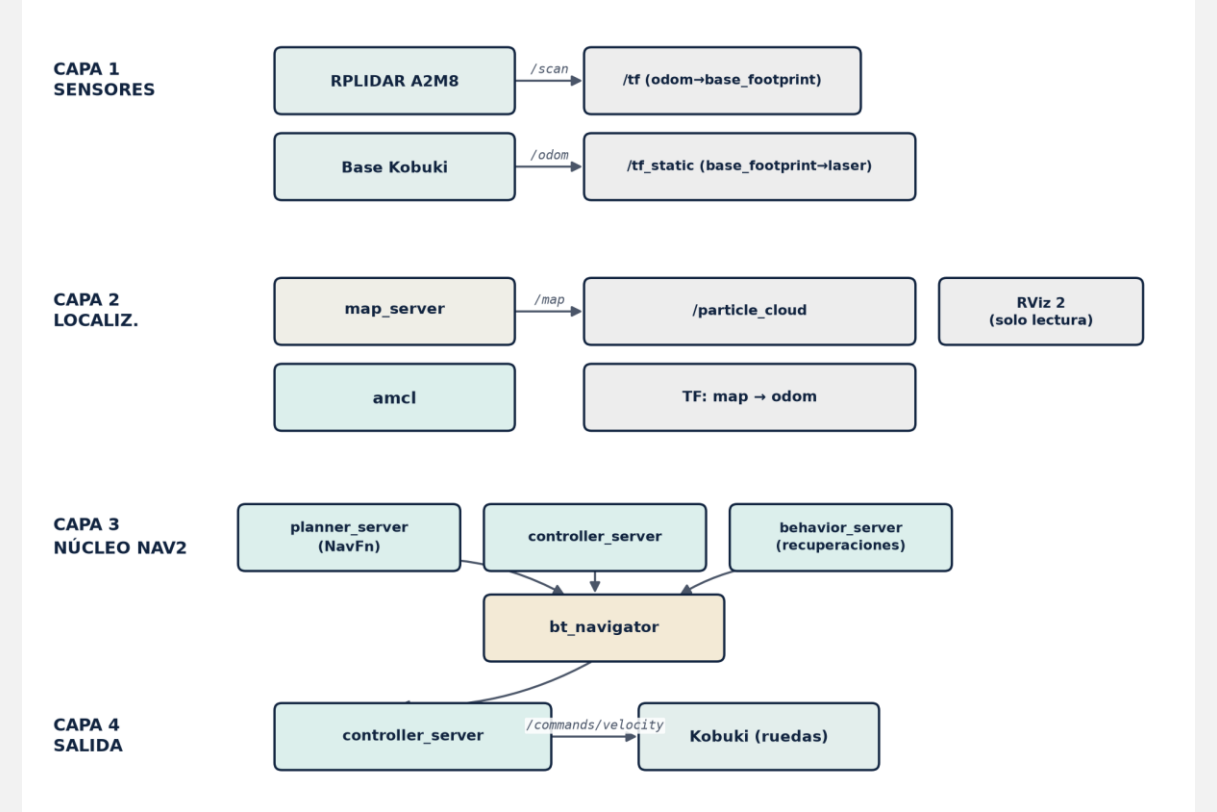
Nota. Elaboración propia a partir del código fuente del proyecto.

2.8 Arquitectura de Nav2 (Fase 3)

La navegación autónoma se organiza en cuatro capas (Figura 2.5): sensores (LiDAR y odometría); localización, donde map_server publica el mapa y AMCL combina barrido, odometría y mapa para estimar la pose (TF map-odom); el núcleo de Nav2 (planner_server, controller_server, behavior_server y bt_navigator, coordinados por un árbol de comportamiento); y la salida, donde controller_server mueve las ruedas.

Figura 2.5

Arquitectura por capas de la navegación autónoma con Nav2.



Nota. Adaptada de la arquitectura de Nav2 descrita en Macenski et al. (2020).

Esta arquitectura se validó por partes: una primera prueba confirmó sensores y localización, y otra posterior también el núcleo de Nav2 y la salida. Con ella se alcanzaron seis de siete objetivos de navegación, incluida la evasión de un obstáculo real no mapeado.

Capítulo 3 — Uso de herramientas de inteligencia artificial

3.1 Declaración de uso

En el desarrollo de este TFM se han utilizado herramientas de inteligencia artificial como apoyo técnico, más o menos como se usaría un buscador avanzado o una herramienta de documentación automática.

Muchas universidades ya reconocen que estas herramientas sirven para aprender y para ganar productividad, siempre que el estudiante mantenga la autoría intelectual: la comprensión del problema, las decisiones de diseño, la validación experimental y la interpretación de resultados.

3.2 Herramientas utilizadas

3.2.1 Asistente de inteligencia artificial conversacional (Claude, Anthropic)

El uso principal ha sido el diagnóstico de errores en ROS 2: errores de compilación (ament_cmake, find_packages()); análisis de registros para distinguir el puerto serie del LiDAR y de la base Kobuki; consultas sobre interfaces de ROS 2; y revisión de código, identificando una doble suscripción no intencionada en el nodo de seguimiento.

Además, en la fase final de redacción el asistente se usó para reformular y recortar párrafos de varios capítulos de la memoria, con el fin de ajustarla al límite de páginas y de mejorar su lectura. El autor ha revisado y validado cada cambio. Las decisiones de diseño (arquitectura, algoritmos y parámetros), las pruebas en el robot, los datos experimentales, su interpretación y las conclusiones son responsabilidad del autor.

3.2.2 Herramientas de autocompletado de código

Durante el desarrollo se ha utilizado el autocompletado del IDE (Visual Studio Code con Pylance) para acelerar código ROS 2 repetitivo, como la declaración de parámetros o la creación de publicadores y suscriptores.

3.2.3 Motores de búsqueda y documentación

Se ha recurrido de forma habitual a la documentación oficial de ROS 2, SLAM Toolbox, Nav2 y MediaPipe como fuente de referencia técnica durante el desarrollo.

3.3 Reflexión sobre el uso ético de la inteligencia artificial

Cuando el asistente me propuso un cambio, lo probé en el robot antes de dejarlo. Una vez señaló una doble suscripción en tracking_node que yo no había visto; la entendí y la corregí yo. Ese es el criterio que he seguido: no incluir nada que no entienda, declarar el uso y responder yo de todo lo que está en el código.

3.4 Estimación del impacto

La Tabla 3.1 resume, para las principales tareas del proyecto, la proporción desarrollada por el autor y el tipo de apoyo recibido de herramientas de inteligencia artificial.

Tabla 3.1

Proporción de trabajo y apoyo de IA por tarea

Tarea	% desarrollado por el autor	Apoyo de IA
Diseño de la arquitectura del sistema	100 %	Ninguno
Algoritmo DBSCAN para detección de piernas	100 %	Depuración
Filtro de Kalman en tracking_node	100 %	Ninguno
Máquina de estados de control_node	100 %	Ninguno
Identificación de errores en la configuración del paquete	60 %	Diagnóstico
Puesta en marcha del robot físico	90 %	Diagnóstico de registros
Documentación técnica (este capítulo)	90 %	Reformulación y recorte de párrafos
Experimentos y validación	100 %	Ninguno

Capítulo 4 — Diario de desarrollo

Este capítulo cuenta el proyecto sesión a sesión, con sus hitos técnicos y las correcciones que fueron saliendo. La Tabla 4.1 recoge la cronología completa. El Capítulo 6 describe cómo quedó cada módulo; aquí lo que se cuenta es el proceso, con sus idas y venidas. La Fase 4 se cerró con la validación del gesto "casa" el 23 de septiembre de 2026 (véase también §7.8 y §7.10).

Tabla 4.1
Cronología de desarrollo.

Fecha	Fase	Hito / corrección
21 may	F1	Primera sesión: corregidos errores de instalación/compilación, conflictos de dispositivo LiDAR/Kobuki, SLAM Toolbox sin activar (lifecycle) y accesos a MediaPipe/scikit-learn sin internet. Seis nodos operativos.
27 may	F1	Segunda sesión: 4 errores corregidos (API ROS 2, señal residual, filtro geométrico de piernas); Kalman de 4→6 estados, gate de Mahalanobis, control PD angular; cámara C270 con HOG.
fin may	F1	Corregido movimiento a tirones (histéresis, predicción del Kalman); primer mapa del laboratorio. Fase 1 completada.
4-17 jun	F2	Ajuste de continuidad de detección; MediaPipe Pose sustituye a HOG (problema de encuadre); gestos de inicio/parada validados.
25 jun	F2	Rumbo de cámara como desambiguador sin par de piernas; DBSCAN reimplementado sobre scipy. 100 % detección sin pérdidas. Fase 2 completada.
8-9 jul	F3	Primera validación en movimiento: saltos y saturación angular, corregidos con filtro de continuidad, gate de Mahalanobis y límite angular (saltos: 1/8 → 1/140).
9-15 jul	F3	Gesto resuelto (cámara SPCA2650); corregidos giro sin control y signo invertido del control PD, verificado con odometría real.
16-21 jul	F3	Redacción capítulos 5-6; corregido sector de evasión invertido; hueco de detección al girar reducido de 2-4 s a ~1,5 s.
22-23 jul	F3	Dos colisiones leves (punto ciego del LiDAR, umbral mal referenciado); corregido el factor de reducción de velocidad y añadida maniobra de rodeo: de 4 contactos a evasión sin contacto.
27 jul	F3	AMCL converge con comandos más largos (latencia DDS); Nav2 alcanza 6/7 objetivos, incluida evasión de obstáculo no mapeado. Fase 3 completada.
29 jul	F3-F4	Vídeo de demostración grabado; confirmadas 2 limitaciones abiertas: evasión no distingue persona/obstáculo, salto de posición tras oclusión prolongada.
ago-sept	F4-F5	Fase 4: cierre del escenario de evaluación y consolidación de métricas. Fase 5: revisión final de la memoria, defensa y etiquetado del repositorio.

Capítulo 5 — Estado del arte

5.1 Introducción y alcance del capítulo

Este capítulo pone en contexto las decisiones del Capítulo 2. Repasa el trabajo previo de cada área (detección con LiDAR 2D, pose por visión, fusión sensorial, gestos, filtros bayesianos, evitación de obstáculos y SLAM/Nav2), sin pretender ser una revisión exhaustiva.

5.2 Sistemas de seguimiento de personas con robots móviles

El seguimiento de personas (person-following) es un problema clásico de robótica de servicio, que combina percepción, estimación de estado y control de movimiento. Con solo LiDAR se aprovecha la

geometría de las piernas y no molesta la luz, pero se confunden las patas de una silla o una columna con piernas. Con solo cámara se saca mucha información (gestos, identidad, orientación), pero cuesta con las oclusiones, la poca luz y un campo de visión limitado. La fusión intenta quedarse con lo bueno de cada una: Sarmento et al. (2024) fusionan tiempo de vuelo con cámara monocular, y Kitamoto et al. (2025) proponen seguimiento con LiDAR 3D y proyección de nube de puntos.

Estos robots suelen moverse en interiores sobre bases diferenciales, como el TurtleBot 2/Kobuki de este TFM. Ahí manda la percepción más que la cinemática. En bases Ackermann de exterior (Li et al., 2026) la maniobrabilidad es limitada y añade restricciones que aquí no tenemos.

El sistema de este TFM se sitúa en la categoría de fusión multisensor (LiDAR + cámara), con el detector LiDAR por clustering heredado de la fase inicial y una vía visual de respaldo añadida en la Fase 2 (§5.5).

5.3 Detección de personas mediante LiDAR 2D (detección de piernas)

La detección por barridos 2D a la altura de las piernas es una línea consolidada: segmentar en clústeres, filtrar por características de una pierna (tamaño, radio, circularidad, curvatura) y emparejar pares plausibles, opcionalmente siguiendo su trayectoria para distinguir marcha de objetos estáticos similares.

Cha y Chung (2020) proponen detección de piernas en un espacio de características 3D con LiDAR 2D; Yao et al. (2023) abordan la diferenciación de múltiples objetivos. Un reto recurrente es la ambigüedad geométrica y las oclusiones parciales. Guerrero-Higueras et al. (2019) exploran redes convolucionales sobre los barridos 2D como alternativa a características hechas a mano.

El nodo de detección (§2.3.1) sigue el enfoque clásico (DBSCAN, filtrado por tamaño y radio, emparejamiento por distancia coherente) y no una alternativa de aprendizaje profundo (Guerrero-Higueras et al., 2019): así no hace falta GPU y se reutiliza el detector ya validado. La ambigüedad geométrica motiva el respaldo con cámara (§5.5), y la pérdida conjunta en los giros es una instancia del problema de oclusión parcial de Cha y Chung (2020) y Yao et al. (2023).

5.4 Estimación de pose humana por visión

BlazePose (Bazarevsky et al., 2020) es una red convolucional ligera. Saca 33 puntos clave del cuerpo a más de 30 fps sin GPU dedicada y es la base de MediaPipe Pose, que es el que usamos aquí. Su extensión GHUM Holistic (Grishchenko et al., 2022) añade landmarks 3D de manos y rostro, aunque este TFM solo requiere hombros y muñecas para gestos y rumbo.

Comparado con HOG, que usaba la versión original, los modelos por landmarks ganan en dos cosas. Por un lado, no necesitan el cuerpo completo en el encuadre, cosa imposible a ~1 m. Por otro, dan puntos anatómicos (muñeca, hombro) que se pueden usar directamente para gestos y orientación, y no una simple caja delimitadora. Por eso se cambió HOG por MediaPipe Pose (§2.3.2).

5.5 Fusión sensorial LiDAR-cámara para detección robusta

La fusión LiDAR-cámara es un área activa. El LiDAR da una geometría precisa pero ambigua; la cámara da semántica, pero sufre con las oclusiones y la iluminación y en monocular no da una distancia fiable. Los trabajos que he revisado van de la visión con LiDAR 360° (Li et al., 2026) al transformador de Minase y Tanaka (2026) para LiDAR e imagen térmica. En los dos la fusión mejora a un solo sensor.

Este diseño usa un esquema más ligero, sin cámara de profundidad: visual_detection_node calcula un rumbo angular desde el punto medio de los hombros (MediaPipe), y detection_node lo usa solo como desambiguador cuando no encuentra un par de piernas válido. Es una fusión tardía asimétrica: el LiDAR sigue siendo la fuente primaria de posición 2D, y la cámara solo desempata cuando la geometría no basta.

Lo hice asimétrico para no depender de calibrar con precisión la cámara respecto al LiDAR: solo uso el campo de visión horizontal y un signo de calibración. A cambio, la cámara se desperdicia cuando el LiDAR ya ve las piernas, y en los giros los dos pierden a la persona a la vez, algo que sigue sin resolverse.

5.6 Interacción humano-robot basada en gestos

El reconocimiento de gestos está muy revisado (Wang et al., 2022). Hoy se tiende a usar landmarks del cuerpo y no clasificadores sobre la imagen entera. Pesan menos y no piden un dataset: el gesto es solo una regla geométrica sobre puntos que ya da el modelo. Hay trabajos con MediaPipe que reportan 94-96 % de éxito en tareas dirigidas por gestos (Zhang et al., 2025).

El sistema sigue ese enfoque basado en reglas. Tiene dos gestos, mano derecha para empezar y mano izquierda para parar, definidos como condiciones geométricas sobre landmarks de MediaPipe Pose, con confirmación por fotogramas y espera entre comandos (§2.3.2). El vocabulario es mínimo a propósito, porque la interacción se limita a iniciar y detener el seguimiento.

5.7 Seguimiento de estado: filtros bayesianos

Los filtros bayesianos (Kalman, EKF, UKF y de partículas) son el enfoque de siempre para seguir personas, y llevan así más de dos décadas (Urrea & Agramonte, 2021). Bellotto y Hu (2010) muestran que el UKF puede dar resultados parecidos a un filtro de partículas gastando mucho menos, lo que interesa en plataformas sin GPU como la de este proyecto. Schulz et al. (2003) abordan el seguimiento simultáneo de varias personas desde un robot móvil con filtros de asociación de datos basados en muestreo.

El nodo de seguimiento usa un filtro de Kalman lineal con estado [x, y, vx, vy, ax, ay] y modelo de aceleración constante (§2.3.3, §6.5). Es la opción más ligera y encaja porque el modelo de movimiento y el de observación ya son lineales: un EKF, un UKF o un filtro de partículas costarían más y solo compensarían si la observación fuese no lineal.

5.8 Evitación de obstáculos y control de movimiento

El Dynamic Window Approach (DWA; Fox et al., 1997) restringe la búsqueda al espacio de velocidades directamente alcanzables, simulando trayectorias a corto plazo y eligiendo la que evita colisiones. Es de los métodos reactivos más adoptados en interiores, y el que emplea Nav2 (§5.9) como controlador local estándar.

El nodo de seguimiento evita obstáculos con una idea parecida: se aleja en ángulo y frena cuando hay algo cerca (§2.3.3). Es una versión simplificada del DWA completo, porque no simula trayectorias y aplica una regla reactiva sobre el sector frontal. Nav2 reserva el DWA completo para la navegación sin seguimiento activo.

5.9 Cartografía y navegación autónoma (SLAM y Nav2)

Para cartografía usamos SLAM Toolbox (Macenski & Jambrecic, 2021), la referencia en ROS 2 (mapeo síncrono y asíncrono, localización sobre un mapa guardado y fusión de mapas). En la Fase 2 sustituyó a nuestro SLAM básico de la Fase 1 (§2.3.7), porque es más robusto y cierra bucles.

Para navegación usamos Nav2, el sucesor del stack de ROS 1 (Macenski et al., 2020), que separa planificación global, control local y percepción, coordinados con árboles de comportamiento y lifecycle nodes (más algoritmos en Macenski et al., 2023). Este TFM solo llega a una demostración mínima: AMCL sobre un mapa guardado y navegación a un objetivo. Nav2 puede bastante más, pero el número de sesiones era limitado (§1.4).

5.10 Síntesis: posicionamiento de este TFM respecto al estado del arte

Ninguna de las piezas es nueva. Todas las técnicas existían; el trabajo consistió en hacerlas funcionar juntas en el NUC, que va justo de potencia. Las decisiones propias más claras son estas:

- DBSCAN reimplementado sobre scipy.cKDTree, porque scikit-learn no era compatible con la versión de Python del NUC (§2.3.1).
- Un esquema de fusión LiDAR-cámara a propósito asimétrico y con poca exigencia de calibración, a diferencia de los más simétricos o basados en aprendizaje profundo de la literatura reciente (§5.5).
- Kalman lineal y no EKF ni UKF: como el modelo de observación de este sistema ya es lineal, no hace falta más (§5.7).

No he inventado ningún algoritmo. Lo que aporta el trabajo es la parte de control_node que decide quién manda sobre la base, el seguimiento o Nav2, para que no se pisen. Eso explica las decisiones del Capítulo 6 y los resultados del Capítulo 7, incluida la unión de seguimiento y navegación en el gesto "casa" (§7.8).

Referencias

Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). *BlazePose: On-device real-time body pose tracking* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2006.10204

Bellotto, N., & Hu, H. (2010). Computationally efficient solutions for tracking people with a mobile robot: An experimental evaluation of Bayesian filters. *Autonomous Robots*, *28*(4), 425–438. https://doi.org/10.1007/s10514-009-9167-2

Cha, D., & Chung, W. (2020). Human-leg detection in 3D feature space for a person-following mobile robot using 2D LiDARs. *International Journal of Precision Engineering and Manufacturing*, *21*(7), 1299–1307. https://doi.org/10.1007/s12541-020-00343-7

Fox, D., Burgard, W., & Thrun, S. (1997). The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, *4*(1), 23–33. https://doi.org/10.1109/100.580977

Grishchenko, I., Bazarevsky, V., Zanfir, A., Bazavan, E. G., Zanfir, M., Yee, R., Raveendran, K., Zhdanovich, M., Grundmann, M., & Sminchisescu, C. (2022). *BlazePose GHUM holistic: Real-time 3D human landmarks and pose estimation* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2206.11678

Guerrero-Higueras, Á. M., Álvarez-Aparicio, C., Calvo Olivera, M. C., Rodríguez-Lera, F. J., Fernández-Llamas, C., Martín Rico, F., & Matellán, V. (2019). Tracking people in a mobile robot from 2D LIDAR scans using full convolutional neural networks for security in cluttered environments. *Frontiers in Neurorobotics*, *12*, Article 85. https://doi.org/10.3389/fnbot.2018.00085

Kitamoto, S., Hiroi, Y., Miyawaki, K., & Ito, A. (2025). Robust human tracking using a 3D LiDAR and point cloud projection for human-following robots. *Sensors*, *25*(6), Article 1754. https://doi.org/10.3390/s25061754

Li, Z., Kou, D., Wang, Y., Li, W., Dong, Z., & Zhang, L. (2026). Auto-Follower: A person-following system for urban Ackermann human–machine collaborative robotics. *ACM Transactions on Autonomous and Adaptive Systems*, Article 3787977. https://doi.org/10.1145/3787977

Macenski, S., & Jambrecic, I. (2021). SLAM Toolbox: SLAM for the dynamic world. *Journal of Open Source Software*, *6*(61), Article 2783. https://doi.org/10.21105/joss.02783

Macenski, S., Martín, F., White, R., & Clavero, J. G. (2020). The Marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 2718–2725). https://doi.org/10.1109/IROS45743.2020.9341207

Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., & Ferguson, M. (2023). From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2. *Robotics and Autonomous Systems*, *168*, Article 104493. https://doi.org/10.1016/j.robot.2023.104493

Minase, Y., & Tanaka, K. (2026). *A dual-stream transformer architecture for illumination-invariant TIR-LiDAR person tracking* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2604.00363

Sarmento, J., Neves dos Santos, F., Silva Aguiar, A., Filipe, V., & Valente, A. (2024). Fusion of time-of-flight based sensors with monocular cameras for a robotic person follower. *Journal of Intelligent & Robotic Systems*, *110*(1), Article 30. https://doi.org/10.1007/s10846-023-02037-4

Schulz, D., Burgard, W., Fox, D., & Cremers, A. B. (2003). People tracking with mobile robots using sample-based joint probabilistic data association filters. *The International Journal of Robotics Research*, *22*(2), 99–116. https://doi.org/10.1177/0278364903022002002

Urrea, C., & Agramonte, R. (2021). Kalman filter: Historical overview and review of its use in robotics 60 years after its creation. *Journal of Sensors*, *2021*, Article 9674015. https://doi.org/10.1155/2021/9674015

Wang, X., Shen, H., Yu, H., Guo, J., & Wei, X. (2022). Hand and arm gesture-based human-robot interaction: A review. In *Proceedings of the 6th International Conference on Algorithms, Computing and Systems* (pp. 1–7). Association for Computing Machinery. https://doi.org/10.1145/3564982.3564996

Yao, H., Peng, J., Liao, Z., Zhao, R., & Dai, H. (2023). Leg detection for socially assistive robots: Differentiating multiple targets with 2D LiDAR. In F. Sun, Q. Meng, Z. Fu, & B. Fang (Eds.), *Cognitive systems and information processing* (Communications in Computer and Information Science, pp. 87–103). Springer Nature Singapore. https://doi.org/10.1007/978-981-99-8018-5_7

Zhang, G., Su, J., Zhang, S., Qi, J., Hou, Z., & Lin, Q. (2025). Research on deep learning-based human–robot static/dynamic gesture-driven control framework. *Sensors*, *25*(23), Article 7203. https://doi.org/10.3390/s25237203

Capítulo 6 — Implementación

6.1 Introducción y alcance

El Capítulo 2 enseña el esquema de nodos y topics. Aquí entro en el código: cómo funciona cada nodo por dentro y por qué lo dejé así.

No todos los ficheros de código están en ejecución: el script de instalación registra siete nodos como ejecutables (§6.2), y varios directorios conservan versiones anteriores no registradas. Aquí se describe solo el código que de verdad se ejecuta; el apartado 6.11 explica el legado.

6.2 Estructura del paquete y gestión de configuración

El paquete es de tipo ament_python, con un directorio por nodo, y registra siete ejecutables: control_node, visual_detection_node, detection_node, tracking_node, user_interface_node, collision_handling_node y slam_node.

Los parámetros ajustables están en un único archivo de configuración, con una sección por nodo, que cargan los ficheros de lanzamiento, así que los umbrales se cambian sin recompilar. Cada parámetro incluye un comentario con su justificación y, en varios casos, la fecha y motivo del último cambio.

El lanzamiento principal arranca cinco nodos: control_node, tracking_node, visual_detection_node, detection_node y user_interface_node. collision_handling_node y slam_node quedan fuera: el primero porque control_node no reacciona aún a su señal (§2.3.5), y el segundo por estar desactivado (sustituido por SLAM Toolbox, Fase 2).

6.3 detection_node — clustering, características de pierna y filtrado temporal

DBSCAN propio sobre un árbol de vecinos

El DBSCAN de este nodo está reimplementado sobre cKDTree de scipy y no sobre scikit-learn, porque no era compatible con la versión de Python del NUC (§2.3.1). Sigue la semántica de siempre: un punto es núcleo si tiene suficientes vecinos dentro de un radio, con consulta por radio y expansión por búsqueda en anchura. El coste computacional es despreciable para los cientos de puntos de un barrido filtrado. El radio de vecindad (0,12 m) y el mínimo de vecinos (4) se calibraron según la resolución angular real del RPLIDAR A2M8.

Filtro geométrico multi-característica de piernas

El nodo extrae ocho características de cada clúster (tamaño, radio, dispersión, anchura, altura, proporción, circularidad, compacidad) y aplica seis filtros secuenciales, combinando el filtro original heredado con características adicionales para reducir falsos positivos (patas de mueble) y falsos negativos (piernas ocluidas). Dos umbrales se corrigieron tras confirmar que rechazaban piernas reales.

El emparejamiento de piernas es un algoritmo voraz de vecino más cercano: para cada clúster sin emparejar se busca el más próximo en un rango coherente (0,05-0,35 m) y se combinan en un

candidato cuyo centroide es la posición estimada. No se usa optimización global (asignación húngara), dado el bajo número de clústeres por barrido.

Filtrado temporal de continuidad y confirmación

Tres mecanismos de filtrado temporal protegen contra falsos positivos: persistencia temporal (una detección debe sostenerse varios barridos para confirmarse, y fallar otros tantos para perderse); un filtro de deriva acumulada, que descarta candidatos cuya distancia a la posición confirmada más antigua supere el desplazamiento plausible; y una confirmación por consistencia, más estricta para el respaldo por cámara (exige confirmación siempre).

El respaldo por fusión: sin un par de piernas válido, el nodo calcula el ángulo esperado en el marco del láser desde el rumbo de la cámara, elige el clúster con menor desviación angular, y exige repetición en varios barridos antes de publicarlo. Esta confirmación es independiente de la de pares, porque un único clúster es una señal más débil.

El nodo invierte el signo de las coordenadas justo antes de publicar, en la frontera entre el marco bruto del láser (delante ≈π radianes) y el convenio estándar de tracking_node (delante en el eje x positivo). Mantener el resto de la lógica en el marco bruto y solo invertir al publicar evita reescribir todo el filtrado, a cambio de leer las variables internas en el marco bruto.

6.4 visual_detection_node — detección dual y reconocimiento de gestos

El nodo admite dos detectores intercambiables: MediaPipe Pose si está disponible, o HOG como alternativa automática (resuelto por control de excepciones de importación). Sin MediaPipe no hay landmarks, así que los gestos no están disponibles en modo HOG. HOG, heredado de la Fase 1 y sustituido como principal por MediaPipe Pose (§2.3.2, §5.4), se conserva como alternativa.

La captura de vídeo se ejecuta en un hilo independiente del executor de ROS 2, descartando fotogramas antiguos del búfer y procesando siempre el más reciente. La publicación está acotada a un ritmo objetivo, midiendo el tiempo de procesado para ajustar la espera.

El reconocimiento de gestos y el cálculo del rumbo reutilizan los mismos landmarks de MediaPipe Pose (hombros, muñecas, caderas). El gesto de inicio se activa cuando la muñeca derecha supera al hombro derecho, y el de parada con la izquierda; el margen es proporcional a la escala del torso, no fijo en píxeles, y cada gesto exige confirmación por varios fotogramas y espera entre comandos. El rumbo se calcula desde el punto medio de los hombros, publicado solo si ambos superan la visibilidad mínima.

6.5 tracking_node — filtro de Kalman, control PD y evasión de obstáculos

Es, junto con detection_node, el nodo con más iteraciones documentadas. La versión activa reemplaza un diseño anterior más simple, sin modelo de aceleración, conservado como código legado (§6.11).

Filtro de Kalman de seis estados con modelo de aceleración constante

El filtro mantiene un estado de seis dimensiones (posición, velocidad, aceleración), frente a las cuatro del diseño heredado, con observación que mide solo la posición. La matriz de transición implementa aceleración constante, con ruido de proceso correlacionado por potencias del intervalo temporal; la corrección usa la forma de Joseph, más costosa pero numéricamente más estable.

Gate de Mahalanobis con confirmación por racha

Cada observación se somete a un gate de Mahalanobis frente al umbral chi-cuadrado al 95 % (2 g.l.): se rechaza solo si supera varias veces ese umbral, y si varias observaciones consecutivas caen fuera, el filtro se reancla a la última, asumiendo una reaparición real. Una observación aislada se ignora; una racha sostenida se trata como reaparición legítima, el mismo patrón que aplican los mecanismos de continuidad de detection_node (§6.3).

Controlador de velocidad angular

La velocidad angular se calcula en cinco pasos: (1) PD con zona muerta ±8°; (2) acoplamiento con la velocidad lineal, reduciendo el giro a alta velocidad; (3) atenuación a corta distancia (de cero al valor pleno según la distancia de referencia); (4) arranque suave el primer segundo y medio; y (5) límite de cambio por ciclo, contra un error grande o un salto puntual de detección.

La velocidad lineal sigue una rampa no lineal en función de la distancia a la persona, más suave que una relación lineal cerca de la distancia de referencia, evitando arranques bruscos cuando la persona se aleja poco de esa distancia.

Evasión de obstáculos reactiva (no un Dynamic Window Approach real)

La documentación original decía Kalman + DWA para este bloque, pero tracking_node no tiene un DWA de verdad, que simularía trayectorias en el espacio de velocidades (§5.8). Ese DWA existe en un fichero aparte que no está registrado (§6.11). El mecanismo activo es reactivo, tipo campo de repulsión: para cada punto del sector frontal ±50° dentro de un umbral (0,40 m, subido desde 0,35 m) se acumula un peso que da más importancia a los obstáculos cercanos y frontales, y el ajuste angular final es la media ponderada.

Factor de reducción de velocidad lineal, corregido el 22 de julio. La fórmula original nunca bajaba la velocidad a cero, y el límite inferior que tenía era código muerto (§4.3, §7.6). Por eso el robot nunca frenaba del todo ante un obstáculo sostenido, y hubo dos contactos leves. La fórmula actual usa la distancia mínima real del sector frontal: una rampa que vale 1,0 en el umbral y baja a 0,0 en un segundo umbral más cercano (0,25 m), con ~10 cm de margen.

Maniobra de rodeo, añadida el 22 de julio. Si el factor de reducción se mantiene bajo un umbral (0,5) durante 1,5 s sostenidos, el nodo ejecuta un giro cerrado de 1,5 s en el sentido ya indicado, seguido de un avance recto corto a baja velocidad. La maniobra se aborta y reinicia el giro si el factor cae de nuevo. Sirve para salir del punto muerto en vez de quedarse parado indefinidamente. Es una primera versión y todavía no está validada de forma sistemática (Capítulo 7).

Es más barato a propósito que un DWA completo: no razona sobre restricciones cinemáticas ni sobre varios pasos futuros. Basta para una evasión reactiva a baja velocidad, pero es una simplificación clara frente al DWA de Nav2 (§5.8-5.9). La etiqueta de la arquitectura ya está corregida.

6.6 control_node — máquina de estados finitos y teleoperación

La máquina de estados (INIT → IDLE ↔ TRACKING, con MANUAL y SHUTDOWN alcanzables desde cualquier estado activo) es una única clase con el estado actual como cadena de texto. Dos mecanismos evitan comportamiento errático: histéresis en la pérdida de detección, que espera antes de volver a espera; y autorización por gesto independiente de la detección (con cámara deshabilitada, se concede automáticamente).

La teleoperación por teclado se ejecuta en un hilo aparte que lee la entrada sin buffering, para capturar teclas sin bloquear el executor, y se desactiva si la entrada estándar no es una terminal interactiva.

Esto hace falta porque una señal de interrupción residual puede quedar en el búfer y leerse como señal inesperada al lanzar el sistema.

6.7 collision_handling_node

Es el nodo más simple: aplica un único umbral de distancia sobre el mínimo del barrido del LiDAR, publicando la señal de colisión solo en los cambios de estado. No está en el lanzamiento principal y control_node no se suscribe todavía a su señal (§6.2). Es infraestructura preparada para Nav2, que sí puede reaccionar con sus comportamientos de recuperación.

6.8 user_interface_node

No implementa percepción ni control: es un agregador de estado. Se suscribe a los topics de estado del resto de nodos y republica marcadores de visualización para RViz 2 (posición estimada, clústeres del LiDAR, marcador del robot), un panel de texto y un diagnóstico periódico en los cambios de modo. También lanza y cierra RViz 2 de forma coordinada.

6.9 slam_node (experimental, desactivado por defecto)

Implementación mínima y propia de SLAM 2D: rejilla de ocupación en memoria, TF estática entre mapa y odometría, actualizada desde el LiDAR y la posición de la persona, sin correspondencia de barridos ni cierre de bucles: un acumulador de ocupación, no un SLAM real. SLAM Toolbox lo sustituyó en la Fase 2 (§5.9) y se conserva desactivado como referencia histórica.

6.10 Despliegue: compilación y sincronización con el robot

El paquete se compila en el NUC con enlaces simbólicos a los ficheros fuente, permitiendo que el executor recoja cambios relanzando el nodo, sin recompilar tras cada edición. Un script de sincronización automatiza la copia de los ficheros más modificados (detección, seguimiento, configuración); el resto requiere copia manual, una limitación real a tener presente en futuras extensiones.

6.11 Código no registrado en el paquete

Varios directorios conservan más de una clase de nodo completa, de las que solo se registra una: el de control conserva dos versiones anteriores; el de seguimiento conserva una implementación real de DWA (búsqueda en el espacio de velocidades y visualización de trayectorias en RViz), no en ejecución; y el de interfaz de usuario conserva una versión anterior no registrada.

Ninguno de estos ficheros es importado por otro módulo: no es código muerto, sino versiones anteriores de la misma clase conservadas en el árbol de trabajo. Lo del DWA importa porque la documentación seguía contando que tracking_node lo usaba, cuando el nodo activo usa el controlador reactivo del apartado 6.5. Eso confundía, y conviene limpiarlo antes de la entrega final.

Capítulo 7 — Resultados y evaluación

Los datos de este capítulo salen de las pruebas en el laboratorio de la UJI, hasta la sesión final del 23 de septiembre de 2026 (§7.8). Varios escenarios tienen pocas muestras, y eso se explica en §7.9. Lo que se queda fuera del alcance del TFM está en §7.10.

7.1 Metodología de validación

Cada toma se graba en el NUC como registro de ROS 2, se convierte a formato tabular y se analiza externamente, generando métricas y cuatro figuras (distancia, ángulo y velocidad frente al tiempo, y trayectoria). Métricas estándar: error de distancia (MAE/RMS), error angular medio, velocidades máximas, % de detección y pérdidas.

7.2 Configuración experimental

- **Robot:** TurtleBot 2 sobre base Kobuki, con unidad de cómputo embarcada ejecutando ROS 2 Jazzy.
- **Sensores:** LiDAR RPLIDAR A2M8 y cámara RGB. La cámara cambió a mitad de la campaña de validación (§1.3): la toma de fusión sin movimiento (25/06) y las tres tomas del 08/07 (apartado 7.4) se grabaron con la Logitech C270 original; todas las tomas a partir del 09/07 (apartados 7.5, 7.6 y 7.7) se grabaron ya con la SPCA2650 que la sustituyó.
- **Entorno:** laboratorio de robótica de la UJI, interior, suelo de baldosas.
- **Distancia objetivo de seguimiento:** 1,50 m en la toma del 25 de junio; 1,00 m en las tomas del 8 de julio en adelante, tras un ajuste del parámetro entre sesiones.
- **Activación del seguimiento:** mediante gesto por cámara en condiciones normales; en las tomas del 8 de julio se empleó un mecanismo manual de activación porque el gesto no era fiable con el encuadre de cámara de aquella sesión.

7.3 Resultado 1 — Fusión sensorial sin movimiento (25 de junio de 2026)

Primera validación del mecanismo de respaldo por fusión LiDAR-cámara, con la base del robot inhibida para poder probar el mecanismo de detección con seguridad antes de validarlo en movimiento.

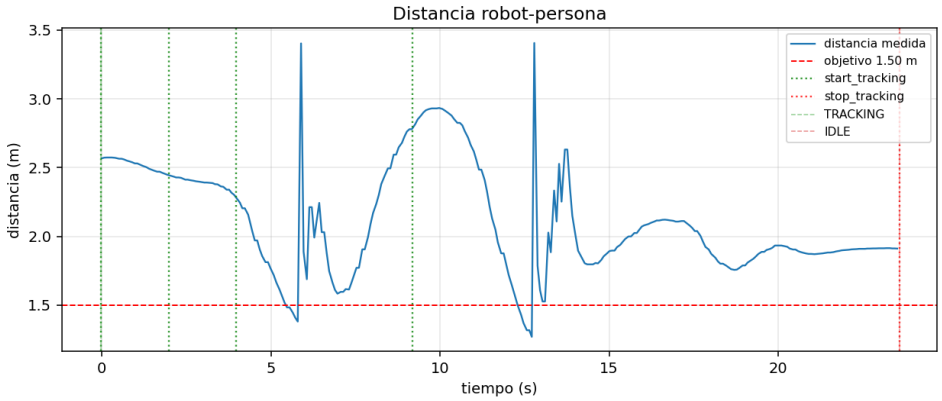
Tabla 7.1
Fusión sin movimiento, 25/06

Métrica	Valor
Duración	23,5 s (285 muestras)
Distancia objetivo	1,50 m
Error de distancia (MAE / RMS)	0,627 m / 0,733 m
Distancia mínima / máxima	1,27 m / 3,41 m
% de tiempo con persona detectada	100,0 %
Número de pérdidas de detección	0
Desviación de rumbo cámara vs. clúster elegido	≈ 6°

Con el mecanismo de fusión activo, la detección fue continua durante toda la toma, pese a que el LiDAR por sí solo no distinguía piernas de forma fiable en esta configuración, que era la motivación original del mecanismo (§2.3.1, §5.5). El error angular medio no es representativo aquí: con la base inhibida, la velocidad angular nunca corrige la orientación.

Figura 7.1

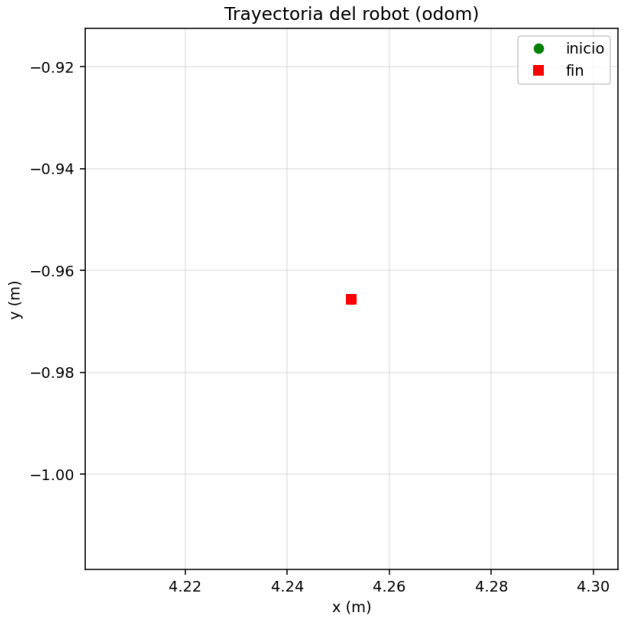
Distancia robot-persona en función del tiempo, toma de fusión sin movimiento (25/06/2026).



Nota. Elaboración propia a partir de los registros de la toma.

Figura 7.2

Trayectoria del robot (odometría), toma de fusión sin movimiento. La base permaneció inmóvil por diseño de la prueba.



Nota. Elaboración propia a partir de los registros de la toma.

7.4 Resultado 2 — Progresión de correcciones de continuidad en movimiento (8 de julio de 2026)

Primera prueba de seguimiento con el robot moviéndose de verdad. La toma inicial reveló saltos de detección y saturación angular casi permanente; las dos tomas siguientes verifican cada corrección por separado (§4.3).

Tabla 7.2

Correcciones de continuidad, 08/07

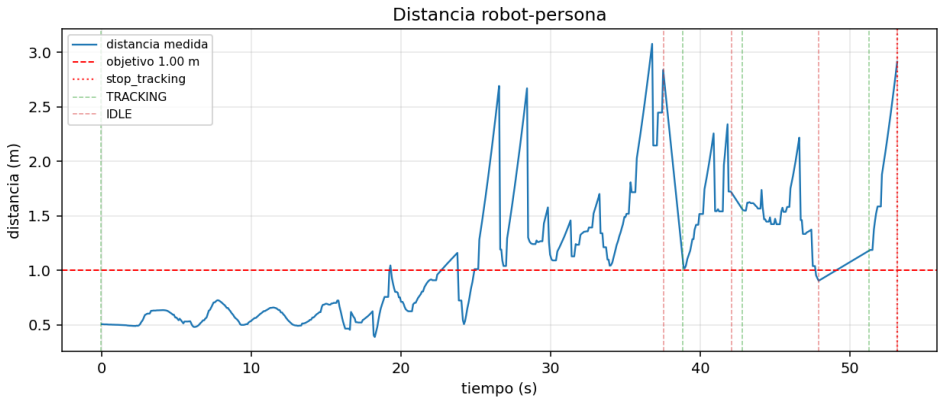
Toma	Duración	% detección	Pérdidas	MAE dist.	RMS dist.	Saltos >0,8 m	Saturación wz
Original (sin corregir)	759,8 s	32,3 %	79	0,519 m	0,935 m	3,5 %	95,7 %
+ filtro de continuidad	249,0 s	71,7 %	43	0,534 m	0,725 m	2,2 %	99,3 %
+ Mahalanobis y límite de wz	53,1 s	82,6 %	13	0,491 m	0,609 m	0,7 %	86,9 %

Las cifras de saltos y saturación se recalcularon el 21 de julio con el pipeline estándar sobre los tres registros originales; el porcentaje de saltos reprodujo con exactitud en dos de las tres tomas y algo más bajo en la tercera, diferencia menor atribuible a la reconstrucción de una metodología antes ad-hoc.

Los tres ajustes encadenados redujeron claramente los saltos de posición implausibles y aumentaron la detección: menos saltos producen un Kalman más estable. La saturación angular no mejora de forma sostenida, porque los ajustes atacan la continuidad de la detección y no la ganancia del controlador angular. El ajuste que sí la reduce (atenuación por proximidad, §6.5) se incorporó más tarde, el 15 de julio.

Figura 7.3

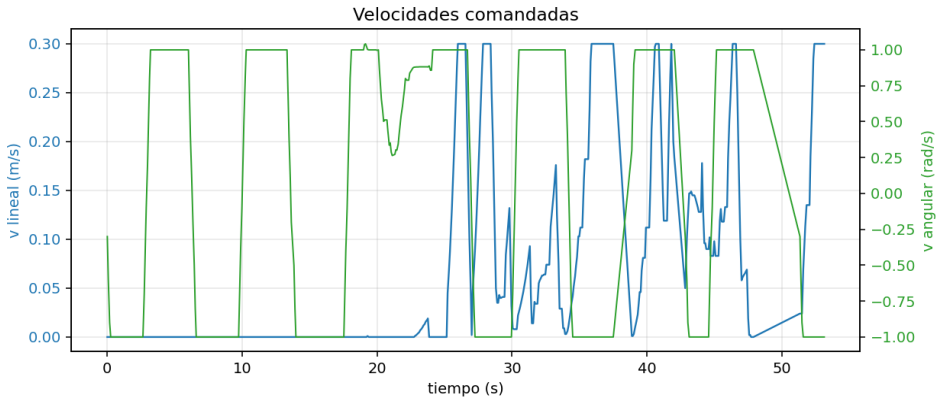
Distancia robot-persona en función del tiempo, toma con los tres ajustes de continuidad aplicados.



Nota. Elaboración propia a partir de los registros de la toma.

Figura 7.4

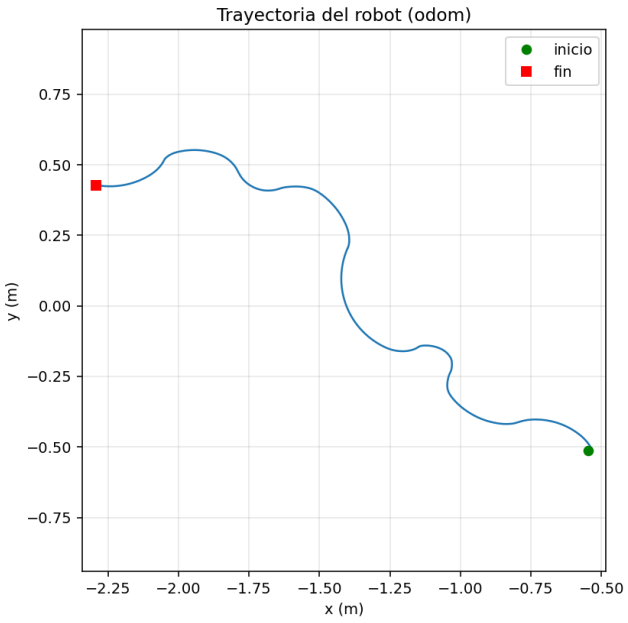
Velocidad lineal y angular en función del tiempo, misma toma.



Nota. Elaboración propia a partir de los registros de la toma.

Figura 7.5

Trayectoria del robot (odometría), misma toma.



Nota. Elaboración propia a partir de los registros de la toma.

7.5 Resultado 3 — Repeticiones por escenario con el sistema estabilizado (22 de julio de 2026)

Con los ajustes del 21 de julio ya en producción, se repitieron entre dos y tres tomas de cinco escenarios sin riesgo de colisión: recta, curva, distancia corta, parada y oclusión.

Tabla 7.3

Escenarios con el sistema estabilizado, 22/07

Escenario	Duración	MAE dist.	Áng. medio	% detec.	Pérdidas	Saltos >0,8 m	Sat. estable
Recta (contaminada)	41,4 s	0,626 m	62,0°	76,2 %	7	6,8 %	13,8 %
Recta (limpia)	25,9 s	0,420 m	26,5°	100 %	0	2,3 %	1,8 %
Curva #1	33,5 s	0,794 m	14,5°	100 %	0	2,8 %	6,8 %
Curva #2 (corta)	12,2 s	0,759 m	23,7°	97,1 %	1	2,0 %	20,8 %
Curva #3	14,3 s	0,976 m	6,2°	100 %	0	0,9 %	0,0 %
Parada	36,5 s	0,326 m	5,9°	100 %	0	0,0 %	0,0 %
Corto #1 (mín. 0,15 m)	32,5 s	0,443 m	32,8°	100 %	0	2,0 %	25,1 %
Corto #2 (corta, mín. 0,42 m)	8,4 s	0,628 m	4,9°	98,8 %	1	0,9 %	0,0 %
Corto #3 (mín. 0,02 m)	18,7 s	0,508 m	46,8°	100 %	0	1,2 %	28,6 %
Oclusión	30,6 s	0,678 m	16,2°	100 %	0	2,5 %	1,5 %

La primera toma de recta quedó contaminada por un estado de seguimiento residual (oscilación ya documentada el 17 de junio); no se repitió tras desautorizar el seguimiento antes de cada toma. Las duraciones cortas de curva #2 y corto #2 se deben a que el gesto tardó en llegar; corto #3 alcanzó 0,02

m, con saturación esperable. Oclusión no produjo pérdida real, gracias al Kalman y al respaldo de pierna única.

7.6 Resultado 4 — Escenario de obstáculo: de los primeros contactos a la evasión sin contacto (22–23 de julio de 2026)

Tras los dos contactos reales descritos en el apartado 4.3 (sesión del 21 de julio), se reintentó el escenario de obstáculo con el umbral de seguridad aumentado. Se realizaron siete tomas en dos sesiones, con dos cambios de código intermedios.

Tabla 7.4
Obstáculo, 22–23/07

Toma	Configuración	Resultado
Obstáculo v3 (mueble)	Umbral de seguridad aumentado, factor de reducción lineal sin corregir	Contacto leve — el factor de reducción nunca llegaba a cero, el robot "arrastraba" sin frenar del todo
Obstáculo v4 (mueble)	Igual	Contacto leve de nuevo, mismo patrón
Obstáculo v5 (mueble)	Factor de reducción corregido: rampa real hasta cero entre el umbral de aviso y el de parada	Sin contacto — encuentro mucho más breve
Obstáculo v6 (mueble + silla)	Igual	Sin contacto — la silla no llegó a activar la evasión en esta toma
Obstáculo v7 (mueble + silla)	+ maniobra de rodeo (giro cerrado y avance corto)	Sin contacto — maniobra disparada cuatro veces, completada con éxito en todos los casos salvo un abandono correcto por seguridad
Obstáculo v8 (mueble + silla)	Igual	Mueble evitado correctamente; contacto directo con la silla, no detectada en absoluto por el LiDAR
Obstáculo v9 (mueble, 23/07)	Sin cambios de código respecto a v7/v8	Sin contacto — segunda repetición limpia del fix, confirmando el resultado en una sesión distinta

El fallo estaba en la fórmula: el factor de reducción nunca bajaba de 0,4 porque el límite inferior no se aplicaba nunca (código muerto). El robot veía el obstáculo y aun así no paraba del todo; por eso rozó dos veces. La corrección basó el factor en la distancia mínima real del sector frontal, con parada dura a partir de un segundo umbral, y se añadió la maniobra de rodeo.

En la toma v8 el robot chocó con la silla, el quinto contacto acumulado. Ya lo temíamos: el LiDAR barre a unos 47 cm y no ve las patas finas de una silla. No repetimos la prueba hasta tener otro sensor, ya sea la cámara RGB-D disponible o un segundo LiDAR a otra altura.

7.7 Resultado 5 — Nav2: localización y navegación autónoma (27 de julio de 2026)

Primera prueba real de la pila completa de Nav2 (objetivo específico 3). La localización con AMCL parecía congelarse tras su primera actualización. El problema era que los comandos de prueba, muy cortos, no llegaban a tiempo por la latencia de descubrimiento de nodos; con comandos más largos convergió sin problemas.

A continuación se probó por primera vez la fase de navegación completa, con siete objetivos enviados desde la interfaz de RViz:

Tabla 7.5
Navegación con Nav2, 27/07

Objetivo	Distancia aprox.	Resultado
1	≈ 3,7 m	Éxito
2	≈ 5,9 m	Éxito
3 (obstáculo real no mapeado)	≈ 5,3 m	Éxito — detectado y esquivado por el costmap local
4	≈ 8,0 m	Éxito
5	≈ 8,1 m	Éxito
6 (cambio de objetivo a mitad de trayecto)	≈ 0,6 m	Fallo — recuperado automáticamente por el gestor de ciclo de vida en unos 2 s
7 (tras la recuperación)	≈ 3,6 m	Éxito

Seis de siete objetivos se completaron (86 %), incluyendo dos trayectos largos de ~8 m y uno con evasión de un obstáculo real no presente en el mapa, esquivado por el costmap local. El único fallo se autorrecuperó con el gestor de ciclo de vida de Nav2, sin repetirse. Durante la sesión vimos que el mapa guardado no se parecía bien al espacio real. Lo volvimos a grabar y lo validamos antes de sustituirlo, así que todos los resultados son ya con el mapa nuevo.

7.8 Resultado 6 — Gesto “casa”: integración de seguimiento y navegación (23 de septiembre de 2026)

Última sesión de laboratorio. La persona hace el gesto “tejado” (las dos manos juntas sobre la cabeza) y el robot deja de seguirla. Después vuelve solo, con Nav2, a una pose “casa” predefinida. Es la primera vez que seguimiento y navegación funcionan en un mismo sistema. Lo que decide qué velocidad llega a la base es control_node (§2.3.4, §6.6).

Tabla 7.6
Métricas del gesto “casa”, 23/09

Métrica	Valor
Vueltas a casa completadas	9 de 11 (82 %)
Tiempo gesto → en casa	15,5 s de media (11-21 s)
Error de llegada	0,27 m / 8° de media (tolerancia Nav2: 0,25 m / 14°)
Cancelación con la mano izquierda	Nav2 confirma en 20 ms, ~7 cm de desplazamiento
Falsos positivos del gesto	0 en ~4 min de gesticulación libre
Tejado reconocido de espaldas	2 de 2 (más lento: la cabeza tapa una muñeca)

Los dos fallos no vinieron ni del gesto ni de la máquina de estados. La localización (AMCL, σ≈0,5 m) puso al robot dentro de la zona de seguridad de un obstáculo del mapa, y desde ahí el planificador no encontró ruta válida. El robot volvió a reposo de forma segura. Es un primer resultado: 11 intentos, en una sola sesión (véase §7.9).

7.9 Limitaciones de los resultados actuales

- **La muestra es pequeña.** Varias filas salen de una sola toma, así que no tenemos variabilidad entre pruebas iguales. Las pruebas de parada y oclusión solo tienen una repetición al cerrar el capítulo.
- **Duraciones y gesto de activación.** Las tomas del apartado 7.4 duran cosas muy distintas, así que las cifras son proporciones dentro de cada toma, no valores normalizados. Además, las del 8 de julio se arrancaron con un mecanismo manual y no con el gesto real, de modo que el objetivo 1 solo aparece en las tomas posteriores al 9 de julio.
- **Nav2 solo tiene una sesión, sin repeticiones.** Así no sabemos si lo que vemos es un patrón o algo puntual. Tampoco hemos mirado a fondo por qué falló la única vez que falló.
- **Objetivo 5: solo hay un primer paso, con muestra reducida. El gesto "casa" (§7.8) junta seguimiento y navegación por primera vez: 9 de 11 vueltas a casa completadas en una única sesión (11 intentos). Los 2 fallos vinieron de la localización (AMCL, ±0,5 m), que dejó al robot junto a un obstáculo del mapa sin margen para que el planificador trazara ruta, y no del gesto ni de la máquina de estados. El guiado a varios destinos, que sería el alcance completo del objetivo, queda como trabajo futuro (§7.10).**
- **Recursos de cómputo compartidos.**El NUC corre a la vez el LiDAR, la cámara con MediaPipe, el seguimiento, SLAM y Nav2, y la interfaz de diagnóstico. En las primeras fases eso daba desconexiones y bloqueos de puerto, y hubo que optimizar bastante el código (puertos duplicados, tasas de publicación, un pipeline de detección más simple; Capítulo 4). Con todo activo, la carga final medida es de ~30 % por núcleo (4 núcleos), sin saturación. Lo que no hemos comprobado es si algún pico puntual influyó en los dos fallos de localización.

7.10 Trabajo futuro

Para el futuro quedan cuatro líneas, fuera del alcance de este TFM: guiar a varios destinos (con un gesto o un código QR por destino, lo que completaría el objetivo 5); una maniobra de salida cuando la localización deja al robot junto a un obstáculo, antes de pedir ruta a Nav2; repetir las pruebas de parada y oclusión con más muestras; y probar el sistema con más personas, entornos e iluminaciones.